

Procedure Calling, Stack Usage, and Memory Layout in MIPS

1 Where to Back Up Data

A procedure needs a place to:

- Back up its arguments, return address, and temporaries if it is a **caller**.
- Back up its caller's saved registers if it needs to overwrite them.
- Place extra data that does not fit in the data registers (temporary and saved registers).

Solution: Use a stack in memory.

2 Memory Layout

- **Reserved:** For operating system.
- **Text:** Program instructions.
- **Static data:** Global variables, e.g., constant arrays/strings, static variables in C.
 - The register `$gp` is initialized to address allowing $\pm$ offsets into this segment.
- **Dynamic data:** Heap, e.g., `new` in C++/Java, `malloc` in C.
- **Stack:** Automatic storage, for procedure data.

3 Local Data on the Stack

- Every procedure has a **stack frame**.
- `$fp` (frame pointer, register 30) points to the start of the procedure's stack frame.
- `$sp` (stack pointer, register 29) points to the top (end) of the stack frame.
- `$fp` is useful when `$sp` changes throughout a procedure.
- For simplicity, stack extensions occur only at procedure entry and exit.

4 Procedure Calling Convention

4.1 At Function Entry

- Allocate stack memory if needed.
- If function is a **caller**, back up `$a_` and `$ra`.
- If function uses `$s_` registers, back up caller's `$s_` registers.

4.2 At Function Exit

- Write return value to `$v_` registers.
- Restore caller's `$s_` registers.
- Deallocate stack memory.
- Return control to caller with `jr $ra`.

4.3 Before Calling Another Function

- Write arguments to `$a_` registers.
- Back up `$t_` registers on the stack if needed.
- Transfer control to callee using `jal`.

4.4 After Calling Another Function

- Restore `$a_`, `$ra`, and `$t_` registers.
- Obtain return value from `$v_` registers.

5 Leaf Procedure Example

C code:

```
int leaf_example(int g, int h, int i, int j) {
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

- Arguments g, h, i, j in \$a0, \$a1, \$a2, \$a3.
- Local variable f stored in \$s0.
- Caller's \$s0 saved on the stack.
- Result returned in \$v0.

MIPS assembly:

```
1 leaf_example:
2     addi $sp, $sp, -4      # Allocate stack space for saving $s0
3     sw   $s0, 0($sp)      # Save $s0 on stack
4
5     add  $t0, $a0, $a1     # t0 = g + h
6     add  $t1, $a2, $a3     # t1 = i + j
7     sub  $s0, $t0, $t1     # f = (g + h) - (i + j)
8
9     add  $v0, $s0, $zero   # Move result to $v0
10
11     lw   $s0, 0($sp)      # Restore $s0
12     addi $sp, $sp, 4      # Deallocate stack space
13     jr   $ra              # Return to caller
```

Listing 1: leaf_example

6 Non-Leaf Procedure Example

C code:

```
int fact(int n) {
    if (n < 1)
        return 1;
    else
        return n * fact(n - 1);
}
```

- Argument n in \$a0.
- Save \$a0 and \$ra on the stack to preserve across recursive call.
- Result returned in \$v0.

MIPS assembly:

```
1 fact:
2     addi $sp, $sp, -8      # Allocate stack space for $a0 and $ra
3     sw   $ra, 4($sp)      # Save return address
4     sw   $a0, 0($sp)      # Save argument n
5
6     slti $t0, $a0, 1      # if (n < 1)
7     beq  $t0, $zero, L1   # branch if not less than 1
8
9     addi $v0, $zero, 1     # return 1
10    addi $sp, $sp, 8       # Deallocate stack
11    jr   $ra               # Return
12
13 L1:
14    addi $a0, $a0, -1      # n - 1
15    jal  fact              # Recursive call
16
17    lw   $a0, 0($sp)       # Restore original n
18    lw   $ra, 4($sp)       # Restore return address
19
20    mul  $v0, $a0, $v0     # n * fact(n-1)
21
22    addi $sp, $sp, 8       # Deallocate stack
23
24    jr   $ra               # Return
```

Listing 2: fact